DayJournal design

This is a map of the source for someone about to change it. The user manual says what the program does; this says where and how. DayJournal is written in Python with PySide6 (Qt 6), like pylg, and uses easyocr for the OCR.

The pieces

File	What is in it
`dj.py`	`main`: starts the window, or the OCR process with `--ocr`
`mainwin.py`	`MainWindow`: actions, menus, the list, search, calendar, tags, notes, saving
`journal.py`	`Journal` and `Entry`: the directory, names and dates, notes, drawings, the tags cache. No widgets
`tags.py`	`cleantags`: from OCR text to tags
`ocr.py`	`readimage` with easyocr, and `serve`, the loop of the OCR process
`ocrworker.py`	`OCRWorker`: runs the OCR process with `QProcess` and reads its answers
`search.py`	`Search`: parses and evaluates search expressions
`viewer.py`	`ImageView`: a `QGraphicsView` with the image, the zoom and the drawing tools
`drawing.py`	the shapes (`LineShape`, `PencilShape`, `CircleShape`, `TextShape`), their JSON form and the undo commands
`drawbar.py`	`DrawBar`: the toolbar for an `ImageView` (tools, color, width, undo, redo, delete)
`tests/`	headless tests; the OCR is faked
`dj.spec`	pyinstaller spec: `DayJournal.app` on macOS

`drawing.py`, `drawbar.py` and the drawing part of `viewer.py` are the same code used in pylg's graphics window; keep them in step when changing one.

Data on disk

Everything is in the journal directory, so it can be copied, backed up or synced as a whole:

File	Written by	Contents
`dayYYMMDD[x].png`, `dayYYMMDDweekprof.png`	you	the images; DayJournal never changes them
`<image>.txt`	`Journal.savenotes`	the notes, plain UTF-8 text
`<image>.drw`	`Journal.savedrawing`	the drawings, JSON (below)
`.pyjou-tags.json`	`Journal.savecache`	`{"version": 1, "images": {name: {"mtime", "raw": [[text, confidence]...]}}}`

All writes go to a temporary file first and then `os.replace` it, so a crash never leaves half a file. An empty notes or drawing file is removed instead of written.

The tags cache keeps the raw OCR text, not the tags, so changing the rules in `tags.py` takes effect at once without reading the images again. An image is read again when its modification time changes.

A drawing file:

```
        {"version": 1, "shapes": [
          {"kind": "arrow", "p1": [1167.4, 1307.5], "p2": [1494.3, 1097.3], "color":
"#e53935", "width": 3.0},
          {"kind": "circle", "rect": [1400.9, 1004.0, 233.5, 140.1], "color": "#e53935",
"width": 3.0},
          {"kind": "pencil", "points": [[10, 20], [12, 24]], "color": "#2e7d32", "width":
3.0},
          {"kind": "text", "pos": [800, 150], "text": "EQH sweep", "color": "#1e63d6",
"size": 22}
        ]}
```

Coordinates are image pixels, so the drawing does not depend on the zoom. `line` is like `arrow`. Unknown kinds are skipped when reading.

Names and dates

`journal.parsename` turns a file name into an `Entry` (date, week profile or not, the extra letter). Weeks start on Monday (`weekstart`). A week profile dated a Sunday is for the week starting the next day; any other one is for the week of its date (`Entry.weekmonday`). `Entry.groupday` is the date an entry is placed at: its own date for a day, the Monday of its week for a profile. `sortkey`, `groupkeys` (year, month and week groups of the list), `Journal.weekof` and `Journal.daysof` all build on these.

The window

`MainWindow` keeps:

- `journal`, the open `Journal`, and `current`, the `Entry` shown;
- `shown`, the entries that pass the search and the Days/Weeks boxes, in order. Moving to the previous or next image, the calendar and First/Last all go through `shown`;
- `items` and `groups`, the tree items of the list, and `expanded`, the keys of the groups open, which survive refilling the list;
- `pending`, the drawings changed and not saved, by image name.

`show_(e)` is the one place an image is shown: it saves the notes, ends the editing of a text box, and loads the image, its notes, tags and drawing (`drawingof`: the pending one, or the saved one).

`applysearch` recomputes `shown` and refills the list (`fillist`). It runs when the search text, the Days/Weeks boxes, or the tags change.

A `QFileSystemWatcher` on the directory calls `reload` a second after a change: it reads the directory again and shows the current image again only if its file changed, so saving the notes doesn't reset

the drawing or its undo history.

OCR

Running easyocr (torch) in a thread of the Qt program deadlocked, and importing it there is very slow under PySide's import hook. So the OCR runs in a separate process: `ocrworker.OCRWorker` starts this same program with `--ocr` (`sys.executable` with `dj.py`, or the app itself when built by pyinstaller), writes the image paths to its standard input, one per line, and reads one JSON line per image back: `{"path", "raw"}` or `{"path", "error"}`. `dj.py` handles `--ocr` before importing Qt.

`MainWindow.startocr` sends the images whose cache entry is missing or old, newest first; `tagged` stores each result, saves the cache every few images, and refreshes the tags and the search. Images asked for while the process runs wait in `ocrpending`.

`tags.cleantags` keeps the words that look like words: it splits on punctuation, drops low confidence text, numbers, dates, month and weekday names, short Spanish and English words, chart chrome (`NOISE`), lowercase runs without vowels and misread capitals (`PtHOG`), and keeps time frames such as `m15`.

Search

`search.Search` is a small recursive descent parser:

```
or   := and ('|' and)*
and  := not+
not  := '!' not | '(' or ')' | word
```

Each word becomes a regex (`wordpattern`): a whole word, ignoring case, with an optional lowercase final `s` and `*` as any word characters. An entry matches when the expression is true over its tags, notes and the text of its drawing (`Entry.drawtext`). A bad expression raises `ValueError`; the search box shows the message and keeps the last good search.

Drawing

`ImageView` puts the image in a `QGraphicsScene` as a `QGraphicsPixmapItem` at z 0, and each drawing as an item above it, all in image coordinates. Zooming changes only the view transform.

- The tool (`settool`) decides what a mouse press does: with **select** the scene handles it (selecting and moving items), and a drag on the image pans (`ScrollHandDrag`); with a drawing tool the press creates a shape and the moves call its `setend` until the release.
- Each change is a `QUndoCommand` on the view's `QUndoStack`: `AddCmd`, `DeleteCmd`, `MoveCmd` (made on release from the positions at the press), `TextCmd` and `StyleCmd`. Commands made after the fact (the shape is already drawn) skip their first `redo`.
- A text box is edited in place; when it loses the focus it calls `ImageView.textedited`, which makes the undo step, or removes it if empty.
- Keys on the view: Del deletes, Esc deselects, S/P/L/A/C/T pick the tool, and the arrows, Home, End and W are sent as `navigate` to the window, unless a text box is being edited.

- `drawchanged` is emitted on every undo stack change. The window then compares `drawingdata()` with the saved drawing and keeps it in `pending` if they differ; the title says *(unsaved)* while `pending` has anything. **Save** writes them all (`savedrawings`); quitting or opening another journal asks first (`keepdrawings`).

The undo stack is cleared when another image is shown, so undo is per image and per visit.

Tests

```
QT_QPA_PLATFORM=offscreen python3 tests/test_journal.py
QT_QPA_PLATFORM=offscreen python3 tests/test_drawing.py
```

They run without a display and without easyocr: `ocrworker.ocrcommand` is replaced by a small script that answers with made up text. `python3 ocr.py image.png` prints what the real OCR reads in an image and the tags made from it, to tune `tags.py`.

Building

`pyinstaller --noconfirm dj.spec` makes `dist/DayJournal.app` on macOS (and `dist/dj` elsewhere). The workflow in `.github/workflows` builds it on GitHub for the three systems. The OCR models are not bundled: easyocr keeps them in `~/.EasyOCR` and downloads them the first time.

The PDFs next to these documents are made with `docs/mkpdf.sh`.

Where to change things

To	Change
accept other image names	`NAMERE` and `parsename` in `journal.py`
keep or drop other words as tags	`tags.py` (`NOISE`, `STOP`, `isword`)
change how searches match	`search.py` (`wordpattern`, the parser)
add a kind of drawing	a shape class with `todict`, `setend`, `small` and `setstyle` in `drawing.py`, `fromdict`, the tool in `ImageView.mousePressEvent` and `drawbar.TOOLS`
add a menu entry	`MainWindow.mkactions` and `mkmenus`
change the list groups	`journal.groupkeys` and `MainWindow.initialgroups`